

UNIVERSITY OF ST ANDREWS

CS4302 COURSEWORK 2

Audio Processing

Author:
150008022

November 2, 2018



Goal

The goal of this practical was to learn through practice various aspects of audio signal processing. Matlab[®] was to be used for the processing.

Tasks

The scripts are separated by task given in the practical specification. Where useful, scripts were created to hold functions that were used repeatedly. To run each practical task in order, the script *practical.m* can be used. Figures and sounds will be generated whilst each script runs. Some tasks were implemented in two ways, and can be identified by the alphabetic letter following the number of the task in the script name.

1 Middle C

Middle C is approximately 261.6Hz, though concert pitch has varied historically. To plot this, a sine wave of the same frequency was generated.

Originally a sampling frequency of $2f$ was used, as Nyquist theorem dictates that the sampling rate must be at least double the highest frequency in the signal. However, MATLAB refused to play at this rate, as the documentation states that only values of $1000Hz \leq Fs \leq 384000Hz$ are accepted by the audioplayer function [1]. Instead, the sample rate used for high quality music was used instead, $48kHz$. Graphing the wave at $1000Hz$ showed a far less smooth wave in the time domain, though it did not affect the frequency domain plot (Figure 1).

To generate ten seconds of audio, the length of a single sample was determined using the equation

$$P = \frac{1}{f}$$

where P is the duration in seconds and f is the sample frequency in hertz. $\frac{10}{P}$ then produced the number of samples necessary to produce ten seconds of audio.

In order to produce the amplitude over time for the given frequency, we first needed to convert the frequency from hertz to radians per second using

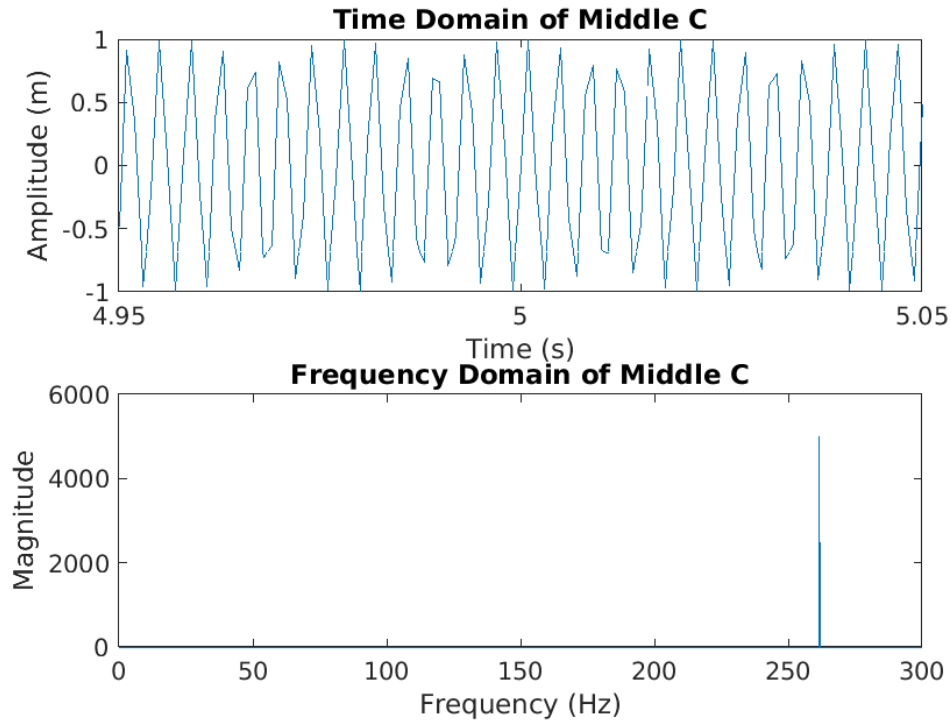


Figure 1: Time and Frequency domain of Middle C at a low sample rate.

the equation

$$\omega = 2\pi f$$

Then we could use the simple harmonic oscillator equation with $\phi = 0$ and $A = 1$ to produce the displacement from equilibrium of the wave at some time t

$$y = A \sin(\omega t - \phi)$$

Matlab greatly simplified the process of producing these values over a range of time through its use of matrices as a standard data type. Out of curiosity, the value of A was changed to see if the resulting sound was louder when played later, and this was confirmed to be true as expected.

The time domain was then graphed, and zoomed in by a factor of 100 so that the actual wave structure was visible (Figure 2). The unit of metres was used for the amplitude, though the perceived amplitude of the sound would

actually be dependant on speakers and external settings when played.

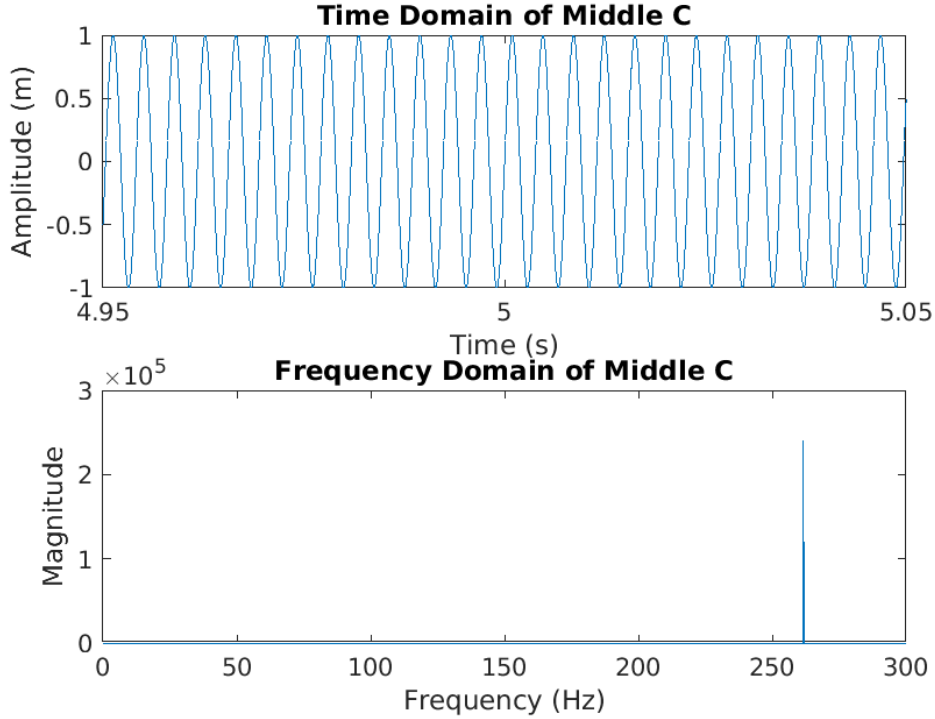


Figure 2: Time and Frequency domain of Middle C at a high sample rate.

Since plotting the frequency domain was such a common task in this practical, it was abstracted into a function script *frequencyDomainPlot.m*. First, the Nyquist limit was calculated using the sample frequency so that frequencies that could not exist within the signal were not plotted. Fast Fourier Transform was then applied to the signal, and shifted so that the zero frequency component was centred in the array. The magnitude axis was produced by taking the second half of values in the fft matrix and taking their absolute value. The magnitude values for negative frequencies, which were not useful for plotting, though they were necessary for inverting the fft later on. The frequency axis was generated by producing evenly spaced values from 0 to Nyquist limit. This produced the bottom graph in Figure 2. Zooming into the graph was enabled using *zoom on*, and this showed that the peak appeared at 261Hz as hoped. Experimenting with various durations of signal also showed that this affected the accuracy of the fft, spreading the

peak over a larger range of frequencies as the duration decreased (Figure 3). The change in time domain plot is only due to the constant zoom being applied to a short range of x-axis values, not an actual change of frequency.

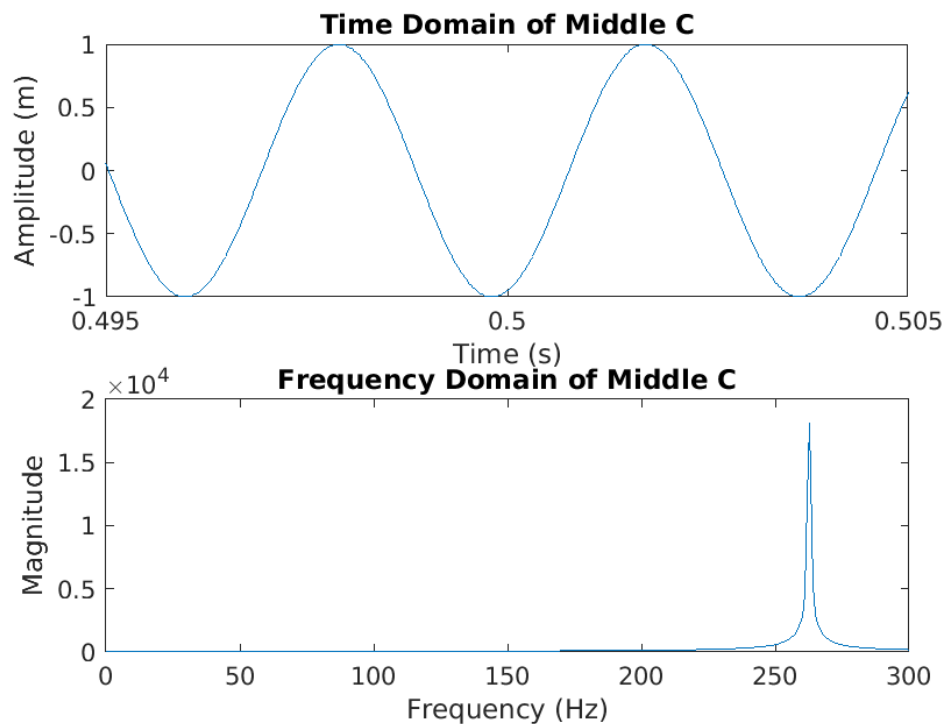


Figure 3: Time and Frequency domain of Middle C at a high sample rate.

The sound produced had a constant intensity and absolutely no variation in pitch. The lack of any other frequencies results in a very unnatural sound.

2 Time Domain

Plotting the time domain followed a similar process as in Task 1. Instead of hard coding a sample frequency however, it was supplied by the audio file itself. From then on, the same process could be applied to the signal as before to produce Figure 4 and Figure 5.

Both plots show that the signals are roughly equally as dense. There also appears to be more noise present in the voice recording, as the 'silence'

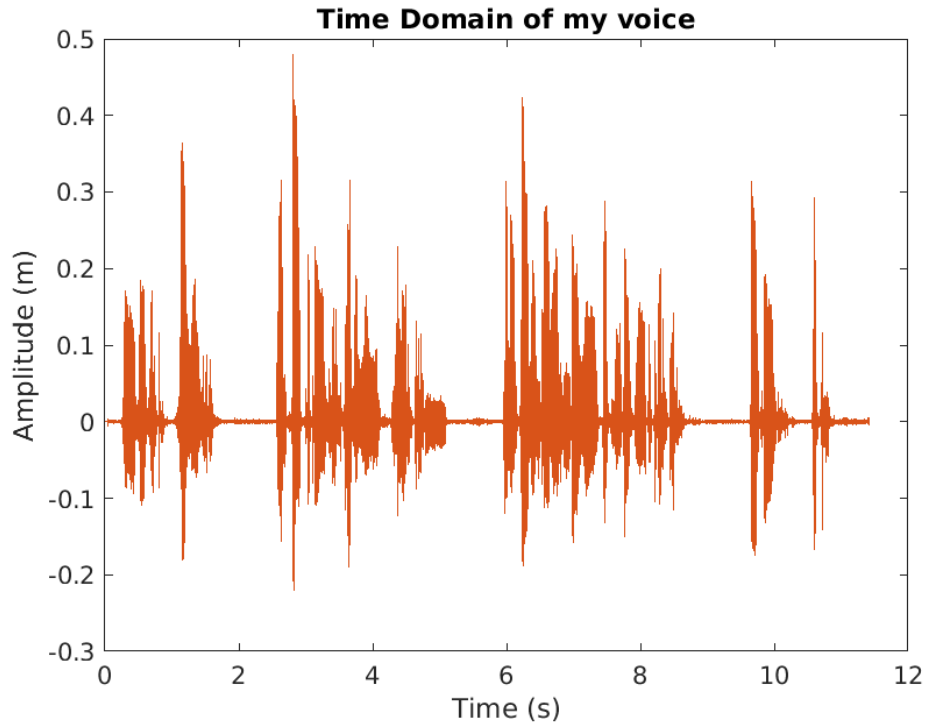


Figure 4: Time domain plot of my voice.

between blocks of speech appears far rougher than the silence between notes in the music plot. There is also more variation in amplitude in the voice plot.

3 Frequency Domain

The same function used for Task 1 was reused here, but this time it produced a more interesting spectrum of frequencies. The high sample used in both files resulted in a large empty space in the original frequency plot, and so a limit was applied so that only frequencies up until $7kHz$ were plotted in Figure 6.

The voice plot appears to have a much more diverse spectrum compared to the music plot. This is likely due to the music only containing frequencies belonging to the notes being played, which are usually constrained within a key. The glissando near the end of the music likely introduced more frequen-

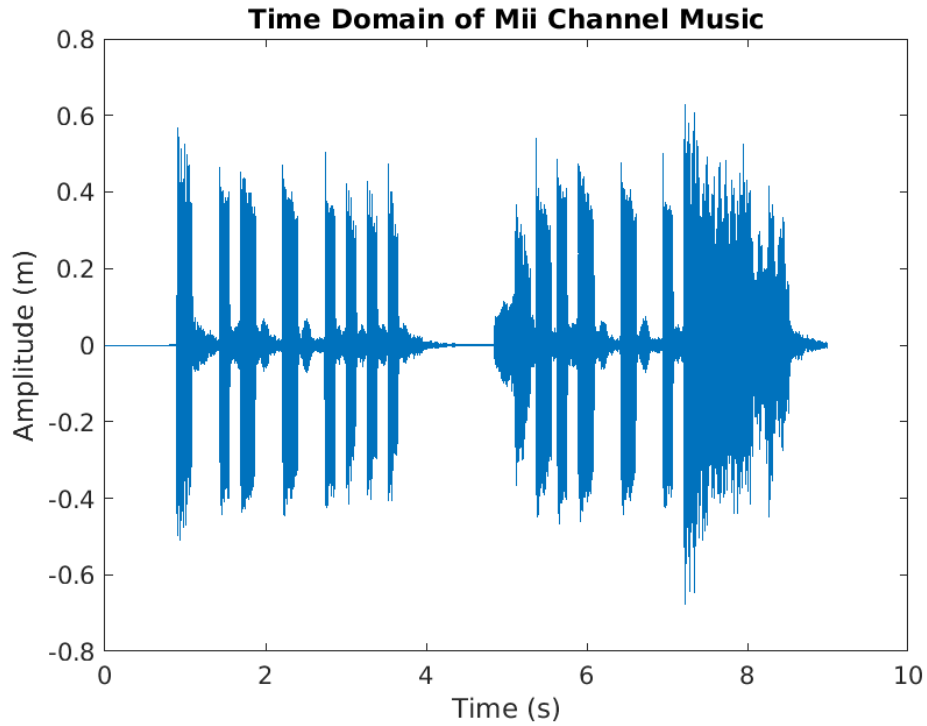


Figure 5: Time domain plot of the Mii Channel Music [2].

cies outside of they key however.

The most prevalent frequency in the voice plot is around $125Hz$, which lies on the upper end of the range of male voice fundamental frequencies. The most prevalent frequency in the music plot is around $386Hz$, which is likely to be a 4th octave G in an equal tempered scale [3].

4 Frequency Filtering

For frequency filtering, I used two methods. The first is my own implementation and the other is using Matlab's *bandpass* function.

For my custom bandpass filter, I took the following steps:

1. Produce the fft of the signal and perform the fft shift.
2. Produce a frequency axis for the signal.

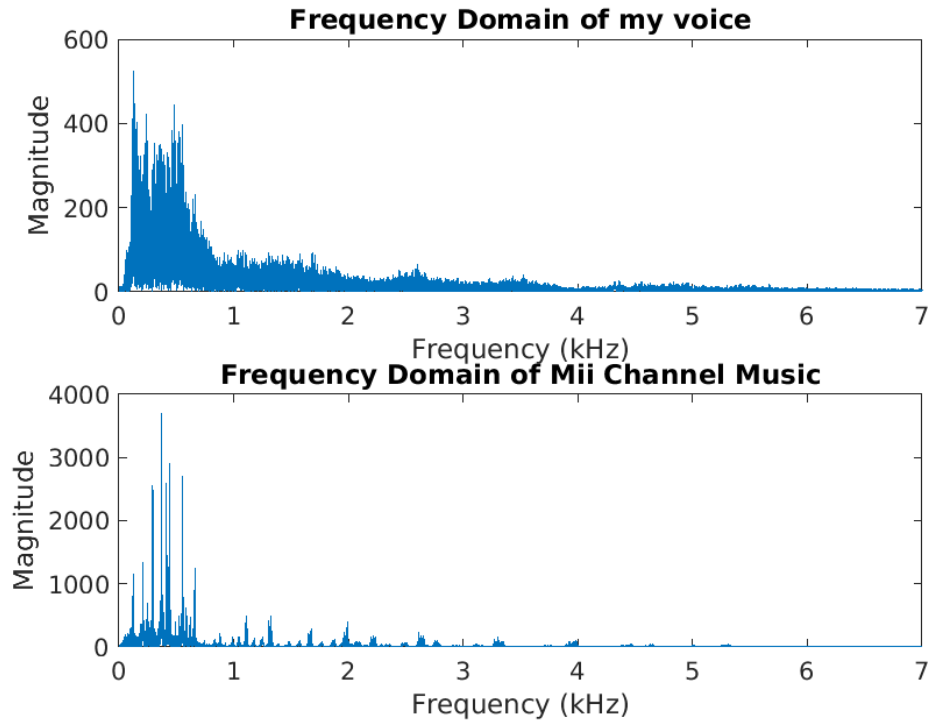


Figure 6: Time domain plot of the Mii Channel Music[2].

3. Zero any values in the frequency axis that are outside the frequency band specified, both in the positive and negative frequencies.
4. Set all other values in the frequency axis to 1.
5. Multiply the fft matrix by the frequency matrix ,effectively zeroing any coefficients that belong to unwanted frequencies.
6. Reverse the fftshift and fft to produce the original signal.

This allowed the same function to be used as both a low-pass (0 to x) and high-pass (x to inf) filter, as well as a bandpass filter.

I applied the same filter to both, allowing frequencies in the range 1 to 2500kHz.

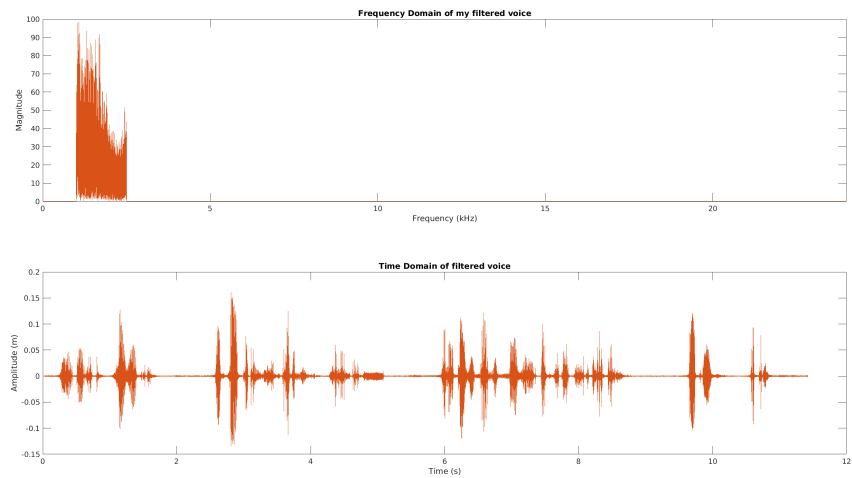


Figure 7: Frequency and time domain plot of my voice after applying the custom filter.

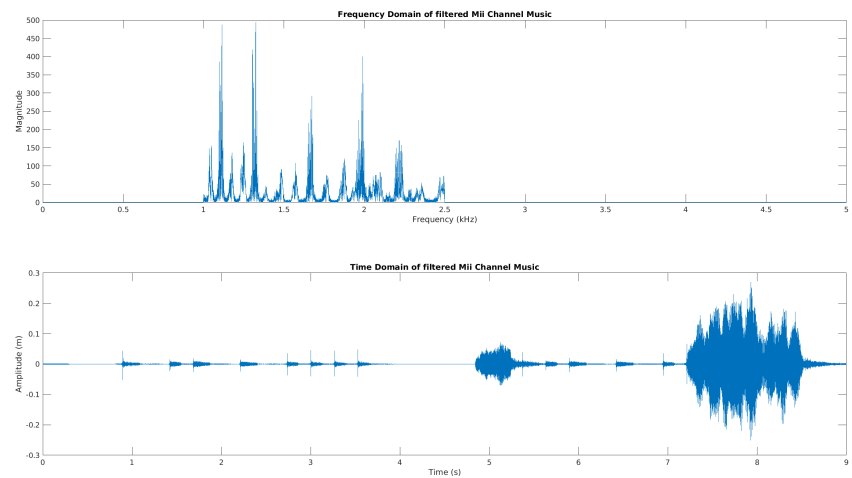


Figure 8: Frequency and time domain plot of music after applying the custom filter.

5 Identifying Noise

6 Removing Noise

7 Extension

8

References

- [1] Matlab documentation. <https://uk.mathworks.com/help/matlab/ref/audioplayer.html>. Accessed: 2018-11-2.

- [2] Kazumi Totaka. Mii channel music, 2006.
- [3] Frequencies for equal-tempered scale. <https://pages.mtu.edu/~suits/notefreqs.html>. Accessed: 2018-11-2.